Palm OS Programming Recipes

Alarm Manager: Enabling Alarms in Applications In order to enable and respond to an alarm in your application, you:

- Set the alarm with `AlmSetAlarm()`. Specify when the alarm should trigger, and which application should be informed when it does.
- Have your application's `PilotMain()` respond to the `sysAppLaunchCmdAlarmTriggered` and `sysAppLaunchCmdDisplayAlarm` launch codes. Depending upon your application, you may also want to have it respond to `sysAppLaunchCmdSystemReset` and `sysAppLaunchCmdTimeChange`.

Note: Palm OS 4.0 introduced the Attention Manager, which simplifies the way that applications respond to alarms. If your application will only be running on handhelds with Palm OS 4.0 or greater, see the "Attentions and Alarms" chapter of the *Palm OS Programmer's Reference and Companion*. **Setting an Alarm** The following steps explain, in greater detail, how to modify your application so that it can set an alarm:

1. In Constructor, add any UI components that are necessary to control the alarm settings.
2. Call `DmGetNextDatabaseByTypeCreator()` to get the card number and database ID of the application that should be notified when the alarm is triggered.
3. Call `AlmSetAlarm()`, supplying the card number and database ID of the application that should be notified along with the time that the alarm should be triggered.

The following code excerpt sets an alarm that will be triggered five seconds after the code is executed:

```
UInt16 cardNo;
LocalID dbID;
DmSearchStateType searchInfo;
UInt32 alarmTime, nowTime;

alarmTime = nowTime = TimGetSeconds();
alarmTime += 5;       //alarm will go off in 5 seconds
DmGetNextDatabaseByTypeCreator(true, &searchInfo, sysFileTApplication,
                        appFileCreator, true, &cardNo, &dbID);
AlmSetAlarm(cardNo, dbID, nowTime, alarmTime, true);
```

Responding to an Alarm When an alarm is triggered, your application is notified with a set of launch codes. The first launch code, `sysAppLaunchCmdAlarmTriggered`, notifies your application that the alarm has occurred; your application need only verify that the alarm is still valid, although it may also want to play a short sound and/or set another alarm. The second launch code, `sysAppLaunchCmdDisplayAlarm`, tells your application that it is now OK to display any appropriate UI.

To respond to an alarm, make the following changes to the responding application's `PilotMain()`:

1. Handle the `sysAppLaunchCmdAlarmTriggered` launch code by playing a system alarm sound and/or triggering the handheld's LED (if so equipped). Set the `SysAlarmTriggeredParamType`'s `purgeAlarm` field to `true` if you won't be displaying an on-screen message and thus don't need to receive the

`sysAppLaunchCmdDisplayAlarm` launch code. Otherwise, set it to `false`.

2. Handle the `sysAppLaunchCmdDisplayAlarm` launch code by displaying a customized form for the alarm. Make sure to first save the form of the currently running application and restore it after the user dismisses your alarm form.
3. If your want your application's alarms to persist across system resets, handle the `sysAppLaunchCmdSystemReset` launch code by retrieving the next appropriate alarm time from your application's stored data (persistent storage) and then setting that time in the system alarm queue with `AlmSetAlarm()`.
4. If your application needs to alter its alarm settings in response to a change in system time, handle the `sysAppLaunchCmdTimeChange` launch code. By default, if an application has an alarm in the queue and the user changes the system clock to a time beyond the set alarm, the alarm immediately goes off when the clock is changed.

Insert the following code into your `PilotMain()` function to implement steps 1 and 2, above:

```
case sysAppLaunchCmdAlarmTriggered:
    AlarmTriggered ((SysAlarmTriggeredParamType *)cmdPBP);
    break;

case sysAppLaunchCmdDisplayAlarm:
    DisplayAlarm((SysDisplayAlarmParamType *)cmdPBP);
    break;
```

Then, implement the `AlarmTriggered()` and `DisplayAlarm()` functions. A sample implementation of `AlarmTriggered()` is shown here:

```
void AlarmTriggered (SysAlarmTriggeredParamType * cmdPBP) {
    cmdPBP->purgeAlarm = true;   // suppresses sysAppLaunchCmdDisplayAlarm

    SndPlaySystemSound(sndAlarm);
}
```

For more information, see the "Attentions and Alarms" chapter in the *Palm OS Programmer's Companion*.